

Supplement S1: Frozen method specification and evidence navigation

LEAP: A Literate-Programming Architecture for Reconstructing
Traceable KPI Knowledge from Enterprise Codebases

Meshaal Obaid Mouawad and Bryan A. Jones

Method-description correction — 23 September 2026

This supplement documents the supplied frozen implementation. It is a static source specification, not a new experiment, an independently adjudicated oracle, or a replacement implementation. Paths are relative to the `JSS_Submtion/` workspace. The software is unchanged. Full source-file hashes and exact symbol ranges appear in `SOURCE_IDENTITY.json` and `METHOD_SOURCE_LOCATORS.csv`; `FROZEN_SOURCE_EXCERPTS.txt` supplies verbatim numbered excerpts. These files are included with the manuscript source as well as with the full workspace.

S1.1 Input, routing, and candidate order

The entry point is `find_kpis_in_directory` in `software/LEAP/bluebook_generator/kpi_extractor.py`, lines 1194–1449. An initial inventory and the subsequent filtered traversal are separate passes. The second uses `os.walk` without sorting directory or filename lists. Hidden/configured directories, configured filenames, and binary extensions are excluded before candidate selection. The default allowed-extension set and exclusions are at lines 5–98; language hints and the dispatcher are at lines 150–182 and 1160–1191. The allowed-extension set is a separate gate from the hint map: an extension’s presence in the latter does not alone ensure that default scanning includes it.

`KPI_SCAN_ALL` accepts `1`, `true`, or `True` to permit otherwise unsupported text-like extensions. Such files use the generic route. `KPI_MAX_FILE_MB` defaults to 2 and supplies a size gate in MiB; the safe reader also has its own 2,000,000-byte limit. Thus the environment setting alone does not remove that reader bound. Empty or unreadable text produces no candidate. The suffix-hint loop returns its first match; exact matching order is part of the frozen implementation.

The language route is attempted first, followed by generic patterns only when no candidate was produced. If neither supplies a candidate, the scanner may construct a rank-0 logic fallback; optional `KPI_FORCE_FILE_FALLBACK` enables a rank-1 fallback. These fallbacks are subsequently withheld by the promotion gate. The helper `_extract_logic_heuristics` also contains a rank-0 constructor, but is not called by this entry point. Its presence is not evidence of an additional exercised path.

Candidate emission order follows extractor pass order, then source-line order within a pass. Examples include: Python comments before function-name candidates; .NET attributes before slash comments, apostrophe comments, and method names; SQL comments before fallback passes; and DAX defined measures before simple measures and KPI comments. Generic extraction has four comment-style passes, followed by its block/bare-line pass. HANA uses SQL-like candidates first and only then a JSON-column fallback. PL/SQL likewise attempts SQL-like candidates before its package-signature fallback. No global sort merges these route-local sequences.

S1.2 Exact ranks and score mapping

Table S1-A covers all detection kinds instantiated in the inspected extractor. The separate CSV records every constructor location, including the second rank-0 logic constructor. A detection kind is a source label, not a measured probability.

Table S1-A. Detection-kind ranks and source locations.

Kind	Rank	Constructor/path	Extractor lines
<code>comment</code>	5	Shared comment helper	598–619
<code>python_function</code>	2	Python name pattern	621–661
<code>csharp_attribute</code>	3	.NET KPI attribute	664–685
<code>method_name</code>	2	.NET method pattern	696–709
<code>sql_comment</code>	3	SQL-family declarations	714–757
<code>create_function</code>	2	SQL function fallback	761–782
<code>create_view</code>	2	SQL KPI-view fallback	784–799

Kind	Rank	Constructor/path	Extractor lines
<code>select_alias</code>	2	SQL metric alias	801–820
<code>plsql_return_fallback</code>	1	PL/SQL last resort	822–851
<code>hana_json</code>	2	HANA column expression	856–904
<code>plsql_function</code>	2	PL/SQL signature	912–933
<code>dax_measure</code>	3	Defined DAX measure	942–960
<code>dax_simple</code>	2	Simple DAX assignment	962–980
<code>dax_comment</code>	3	DAX KPI comment	982–997
<code>abap_report</code>	2	ABAP REPORT pattern	1000–1022
<code>iec_block, iec_brace</code>	3	Structured Text blocks	1025–1069
<code>generic_block, generic_iec</code>	3	Generic block forms	1094–1142
<code>generic_bare</code>	2	Bare KPI declaration	1144–1154
<code>logic_fallback</code>	0	Uncalled helper / scanner fallback	1073–1092; 1308–1340
<code>forced_fallback</code>	–1	Optional scanner fallback	1343–1369

The gate and score function also recognize `empty_file_fallback`, but the inspected file does not instantiate that kind. Rank defaults must therefore not be used to imply a demonstrated route for it.

The score function (lines 1418–1438) starts with $3 \mapsto 95$, $2 \mapsto 85$, $1 \mapsto 75$, $0 \mapsto 60$, and $-1 \mapsto 50$; all other ranks map to 70. It then floors the score at 90 for `comment`, `sql_comment`, `dax_measure`, `csharp_attribute`, and `iec_block`. It caps `forced_fallback` and `empty_file_fallback` at 55. Consequently, rank-5 comment candidates receive 90, not 95. Selection rank and displayed evidence score serve different functions. This specification preserves the frozen mapping rather than replacing it with a monotonic or calibrated scale.

S1.3 Promotion predicates and context boundaries

`evaluate_candidate_promotion` (lines 412–590) returns at the first applicable branch. The following order is significant:

1. Withhold scenario fixtures during an ordinary repository scan; the dedicated scenario-directory scan is an explicit exception.
2. Reject matching invalid/test/mock/debug names; empty, shorter-than-three-character, or generic `kpi` names; embedded newlines/code keywords; names over 120 characters; and names containing braces or semicolons.
3. Compute explicit-marker, formula-marker, business-metadata, and calculation predicates. Reject strict helper-name matches lacking an explicit KPI marker.
4. Withhold weak kinds: `logic_fallback`, `forced_fallback`, `empty_file_fallback`, and `plsql_return_fallback`.
5. For CSV, require explicit KPI and Formula evidence and no bad-formula artifact. This branch returns before the general explicit/inferred branches.
6. For explicit candidates, require at least one of a formula marker, calculation evidence, or business metadata.
7. For the nine medium inferred kinds (Python function, .NET method, SQL alias/function/view, PL/SQL function, HANA JSON, simple DAX, ABAP REPORT), require business context, business-shaped formula structure, and executable-calculation evidence. Python/function-method kinds additionally require local return arithmetic.
8. For other kinds, accept business context plus calculation structure, or explicit metric-language wording plus calculation structure. Otherwise retain an Extraction Review reason.

All predicates are textual rules. They are not an AST-level proof that an expression is executable or that a named aggregate is a business KPI. Table S1-B points to the precise tests; the supplied excerpts contain their exact regular expressions and vocabulary.

Table S1-B. Predicate definitions in `kpi_extractor.py`.

Predicate	Frozen criterion	Lines
Business marker	A token belongs to <code>BUSINESS_KPI_MARKERS</code> , or a marker of at least four characters occurs in compact normalized text.	259–291; 321–329
Formula shape	An arithmetic symbol and an aggregate-call, metric-alias, or business-marker cue occur after string-literal stripping.	332–342
Formula marker	Anchored labels include Formula, Business Formula, Calculation, Computes, Body Computes, and Body Hint.	345–351

Predicate	Frozen criterion	Lines
Business metadata	Anchored objective, unit, owner, source/reporting-system, usage, or input labels.	354–360
Calculation	Supported return-arithmetic, SELECT/arithmetic/AS, aggregate/NULLIF call, or assignment-arithmetic text.	363–370
Python return gate	Local return line contains an arithmetic operator after string stripping.	373–375
String stripping	Triple- and single/double-quoted literal patterns are replaced before the preceding math checks.	378–384
Bad formula artifact	N/A, slash-separated word forms, or long hexadecimal forms. Used in the CSV gate.	387–393
Explicit kind	<code>comment</code> , <code>sql_comment</code> , <code>dax_measure</code> , <code>csharp_attribute</code> , <code>iec_block</code> ; otherwise matching KPI text is required.	484–489
Exact name rejections	Ordered invalid-name expressions, length/content checks, and strict blacklist.	293–307; 441–501

Context windows. The shared `_window` uses up to 10 preceding and 20 following lines, plus the anchor line. .NET attributes and methods use 5/60; SQL view candidates use 5/120; SQL alias candidates use 3/80. PL/SQL explicit comments use 0/160. SQL comments in other dialects and SQL function fallbacks use `_expand_forward_statement`: up to five preceding lines and at most 200 lines from the anchor, stopping at a trailing semicolon at nonpositive tracked parenthesis depth, a lone GO, a lone slash, or end of file. Exact bounds and terminator expressions are at lines 202–244 and their call sites in the locator index.

HANA JSON fallback constructs `SELECT<expression>AS<name>` and assigns anchor 1; it does not recover an original source span for that constructed text. The PL/SQL-return fallback retains the whole file, then fails the weak-kind promotion gate. Scanner logic/forced fallback contexts keep at most the first and last 4,000 characters when the input exceeds 8,000 characters; those kinds are also withheld. Context windows are neither full dependency slices nor guarantees that all null/guard behavior is represented.

Dossier-stage context. These are extraction-time windows. In `main.py`, `create_rst_file` reloads the source file when readable (1469–1488), otherwise using captured context. Python then passes that text and its anchor to `_extract_function_block` (506–558; 1490–1501); the helper retains the original text when no function is located. Other formats use all available text (1502–1504). Thus the formula parser need not receive the original extraction window, and declaration comments may remain in its input.

S1.4 Normalization, duplicate retention, and residual ties

`_normalize_name` (247–256) strips surrounding whitespace and one matching outer pair among brackets, parentheses, single quotes, and double quotes. Otherwise it strips outer quote characters; underscore/whitespace runs collapse to one space. It does not lowercase the result.

After promotion, within-file deduplication retains the first candidate for each exact name; subsequent candidates with that name are discarded before rank selection (1393–1401). Selection then uses `max` over (rank, readable-name flag), where the flag tests for a space, parenthesis, or square bracket (1407–1416). Residual ties retain the first encountered candidate. The cross-file exact-name set likewise keeps the first selected occurrence (1445–1447). No sorting step makes this order-independent.

DAX performs additional earlier lowercase-name suppression in its defined-measure/simple-measure/comment passes (936–997). The comment pass consults that set but does not add new comment names to it; final exact-name deduplication still applies. HANA’s JSON fallback keeps the earlier column on equal local scores. These route-local decisions precede the common selection procedure and must not be silently replaced by a universal case-insensitive deduplicator.

S1.5 Formula preparation, conflict rules, and rendered status

`generate_formula_from_code` in `bluebook_generator/main.py` (561–1428) prepares formula-like text, not exclusively executable expressions. `extract_calculation_hint` (1101–1135) scans comment labels Formula, Business Formula, Body hint, Body computes, Computes, and Calculation. In encounter order, invalid candidates are skipped; the first math-like hint is returned. `extract_result_name_and_expression` gives a suitable hint precedence (1202–1208), then tries NPS and structured-expression rules (1212–1218), followed by SELECT, assignment, return, and fallback paths. A hint name without an expression does not terminate extraction. The main call uses the unfiltered context (1408–1411); construction of cleaned lines at 1185–1191 does not remove comments from this call.

The normalization helper (648–663) removes selected SQL wrappers for display, not for semantic equivalence. Explicit LaTeX preservation/normalization is at 250–489. Declared text can also originate in `parser.py` (4–12). The downstream context transition in S1.3 precedes preparation of `programmatic_formula` (1490–1504). A formula-like `formula_description` is processed separately (1614–1661); these two paths can reuse declaration evidence rather than establish independent executable and declared formulas.

Exact display-quality rule. `_formula_html_quality` (1632–1651) substitutes an empty string for falsy input and returns 0 only when that string is empty or contains the case-sensitive literal `See code context`. Otherwise it starts at 2; it subtracts 1 once if any of `Placeholder`, `= 0.0`, `Nullif`, `\text{Count}`, `\text{Cast}`, `\text{Date Difference}`, `\text{Number}`, or `\text{Return}` occurs; it adds 1 once if `\frac` or `\times` occurs. The note emitted by `_missing_formula_html` (464–470) has none of these markers, so it scores 2, not 0. A literal `Placeholder` alone scores 1. This is a display-selection heuristic, not a formula-validity score.

The declared display is preferred when its quality is at least the programmatic display’s quality, or a mismatch is found (1729–1730). Explicit LaTeX in the record has a later normalization/display path (1740–1764); selected template output and fallbacks are at 2690–2805. Display preference does not establish semantic authority.

The mismatch predicate (1653–1730) compares addition-or-subtraction, division, and multiplication families. Programmatic HTML contributes operator cues (1662–1671). A separate textual scan skips blank lines and lines starting with `--`, `#`, `//`, `'`, or `*`, removes arrow-type symbols, and collects operators (1672–1692). Their union is `evidence_ops` (1693). A formula-like declaration with positive display quality and operator symbols triggers mismatch if one declared family is absent from that union (1694–1705). Skipping comments in the textual scan does not remove declaration-derived operators already present in the programmatic display. Declaration reuse can therefore suppress a mismatch. The cues are not invariably independent executable evidence. The Gross Margin Python example still has the stated mismatch: its function-scoped ratio lacks the declared addition/subtraction family. The predicate also cannot generally distinguish addition from subtraction, arbitrary variable identities, operand order, or whole-program equivalence.

The rendered review label uses ordered conditions (2631–2690): a formula mismatch, governance conflict, audit note, or compliance alarm yields *Needs Review*; otherwise an approved override yields *Validated*; otherwise a numeric score below 75 yields *Needs Review*; otherwise *Validated*. Malformed scores fall back to 0. An override does not bypass the conflict branch. This label differs from intermediate governance status (1584–1606) and the prepared-record harness’s derived status; none is external certification.

Token helpers are at 2228–2478. The token strip is not a grouping-preserving equation; formal mathematics and captured source remain separately inspectable.

S1.6 Verification layers and available evidence

`test_app.py` contains 18 test functions (14–404). The scenario module contributes one suite-level test, giving 19 test functions across the two modules, consistent with the archived pytest result. The 38 fixture inputs are cases inside the suite-level function, not 38 additional pytest functions. `TEST_COUNT_SOURCE_AUDIT.json` records the static AST inventory. Two candidate-promotion functions (243–295), two MATE functions (298–334), and two ISO-template functions (337–404) are included in the 18 application functions, not separate validation populations. The exact test-to-function locations are in the locator CSV. These are source mappings, not newly measured coverage percentages.

The 38-scenario harness constructs prepared records, invokes detail/governance processing, and checks status, review routing, signals, and alarm metadata (`data/validation/test_scenario_fixtures.py`, 58–254). It parses the expected-formula field but does not assert formula equality. The suite is not a repository-discovery or formula-fidelity experiment. No tests, fixtures, or runtime software were modified or rerun to prepare this supplement.

The full review workspace contains:

- `software/LEAP/`, including the actual fixtures, tests, sample project, generated pages, and runtime source.
- `evidence/HISTORICAL_FIXTURE_VALIDATION_AUDIT.md` and the historical processed inventories.
- `evidence/02_runtime/QA-003_JSS_MATE_ISO_Traceability_2026-09-20/`, including its recorded commands and summary.
- The RR-004, RR-005, and RR-006 archived records and their existing interpretation boundaries.
- The existing Sprint 7 scenario transcript at `reproducibility/sprint7/scenario_run.log`, retained under its original identity.

QA-004 transfer status. The author supplied a report of 38/38 passing with exit 0 on 21 September 2026, plus command, timestamp and hashes. This distribution preserves that report in `evidence/reported_runs/QA004_AUTHOR_SUPPLIED_SUMMARY.md`. The original QA-004 directory containing the raw stdout/stderr and environment receipt has not been supplied here. The report is not reconstructed into a fictitious raw bundle, and an existing transcript with the same stdout hash is not relabeled as that execution. The packaged QA-003 evidence supports the narrower test statements retained in the manuscript. The later report is not counted as an independent sample or as new primary evidence.

The workspace `README.md` and `CURRENT_AUTHORITY.md` identify the only current manuscript source and build command. Previous manuscripts remain under history. The smaller source ZIP is for rebuilding and includes this supplement and source excerpts; it must not be mistaken for the full research-artifact workspace.